

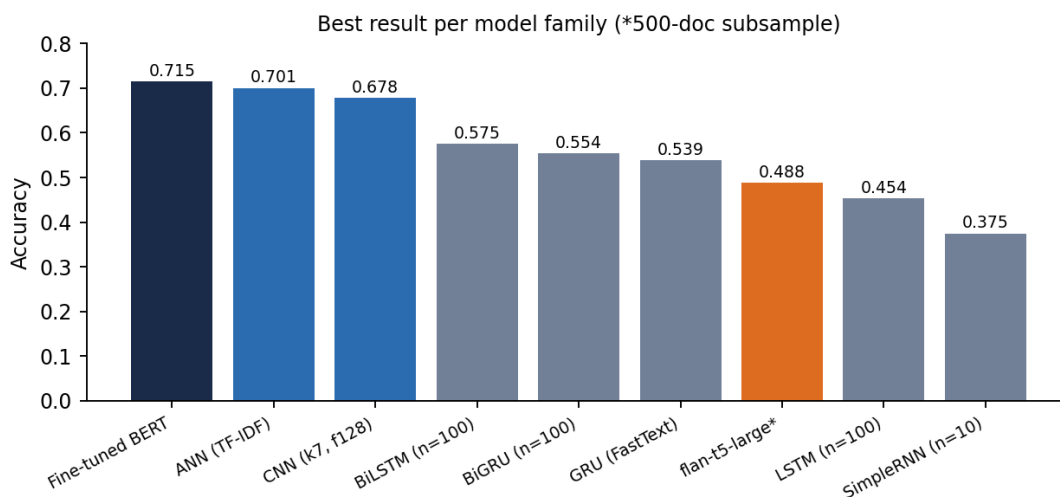
Progressive Text Classification on 20 Newsgroups

Test report: 32 experiments, from a simple neural network to fine-tuned BERT and prompted LLMs. Prepared 28 September 2026 from the Kaggle notebook results.

Fact	Value
Task	Sort a newsgroup post into 1 of 20 topics (chance level = $1/20 = 5.0\%$)
Data	18,846 posts = 13,192 train + 2,827 validation + 2,827 test (70/15/15, one fixed split for all models)
Best result	Fine-tuned BERT: 71.5% accuracy (2,022 of 2,827 test posts correct), macro F1 0.693
Strongest simple model	ANN on TF-IDF: 70.1% (1,981 of 2,827), only 41 posts behind BERT
Best LLM (no training)	flan-t5-large zero-shot: 48.8% on a 500-post subsample
Hardest topic	talk.religion.misc: BERT found 1 of 94 posts (fewer than a random guesser would, about 4.7)
Worst run	SimpleRNN reading 100 tokens: 6.5%, essentially chance

Headline findings

- Fine-tuning BERT gave the top score, but only **1.45 points (41 posts)** above the TF-IDF network. On a 2,827-post test set the 95% sampling margin is about ± 1.7 points, so one run cannot prove BERT is truly better.
- Adapting all of BERT to the task gained **8.3 points** over using BERT frozen (71.5% vs 63.2%).
- Borrowed word knowledge (Word2Vec, FastText) did not lift the CNN; it helped the GRU only with FastText (+3.0 points).
- Recurrent models improved when they read more tokens ($n = 5, 10, 100$), except SimpleRNN, which collapsed at 100.
- Small prompted LLMs never saw the training data: 48.8% (flan-t5, zero-shot) and 10.6% to 23.0% (Qwen2.5-1.5B).



1. What was tested and how

Item	Detail
Dataset	20 Newsgroups, 20 topic groups (comp.*, rec.*, sci.*, talk.*, religion, misc.forsale)
Cleaning	Lowercase; punctuation and digits removed; stopwords and 1-letter tokens removed. Same cleaned text for every model except that the LLMs and BERT tokenizers then read it as given
Split	Stratified 70/15/15, random seed 42, saved and reused; validation used for tuning, test used only for final numbers
Metrics	Accuracy, macro precision, macro recall, macro F1, confusion matrix for every run
Sequence input	Vocabulary 10,000 words; CNN reads up to 200 tokens; RNNs read n = 5, 10 or 100 tokens
"N-gram n = 5, 10, 100"	Interpreted as the number of tokens each RNN reads (context window), not as word-phrase features
LLM sample	500 test posts, 25 per class, fixed seed, identical for every LLM run
Hardware	Kaggle notebook with two GPUs; TensorFlow for ANN/CNN/RNN, PyTorch for BERT and LLMs

Real-life reading of the error bars

- Test set of 2,827 posts: 95% margin about +/-1.7 points at 71% accuracy. Differences under about 2 points between two full-test runs are not reliable.
- LLM subsample of 500 posts: margin about +/-4.4 points at 49% accuracy. Small LLM differences should not be over-read.
- Every figure comes from a single training run per model (no repeated seeds).

2. Baseline: ANN on TF-IDF (Task 2)

Each post becomes a 10,000-dimension TF-IDF vector (word importance), fed to a small fully connected network. Result: accuracy **0.7007**, macro precision 0.7084, macro recall 0.6925, macro F1 0.6961. This is a strong baseline because topic is largely decided by which words appear, and TF-IDF keeps the exact identity of every word.

3. 1D CNN (Task 3)

A convolution slides a window (kernel) over the word sequence and keeps the strongest pattern per filter. Three settings were tried on the same 200-token input:

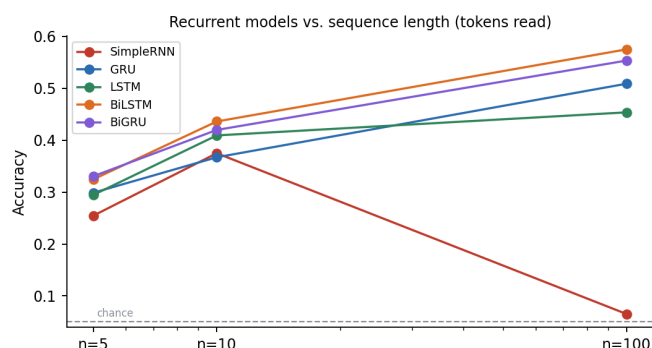
Model	Kernel	Filters	Accuracy	Macro F1
CNN_k3_f64	3	64	0.6654	0.6572
CNN_k5_f128	5	128	0.6611	0.6567
CNN_k7_f128	7	128	0.6778	0.6719

The largest kernel was best, but kernel size and filter count changed together in the second and third runs, so the gain cannot be assigned to one factor. All CNNs stay 2.3 to 4.0 points below the TF-IDF ANN.

4. Recurrent networks (Task 4)

Five recurrent types were run at three context lengths (15 runs). A recurrent network reads tokens one after another and keeps a memory; longer reading means more topic words seen.

Model	n = 5	n = 10	n = 100
SimpleRNN	0.2547	0.3750	0.0651
GRU	0.2989	0.3672	0.5090
LSTM	0.2947	0.4093	0.4538
BiLSTM	0.3247	0.4362	0.5752
BiGRU	0.3307	0.4199	0.5536

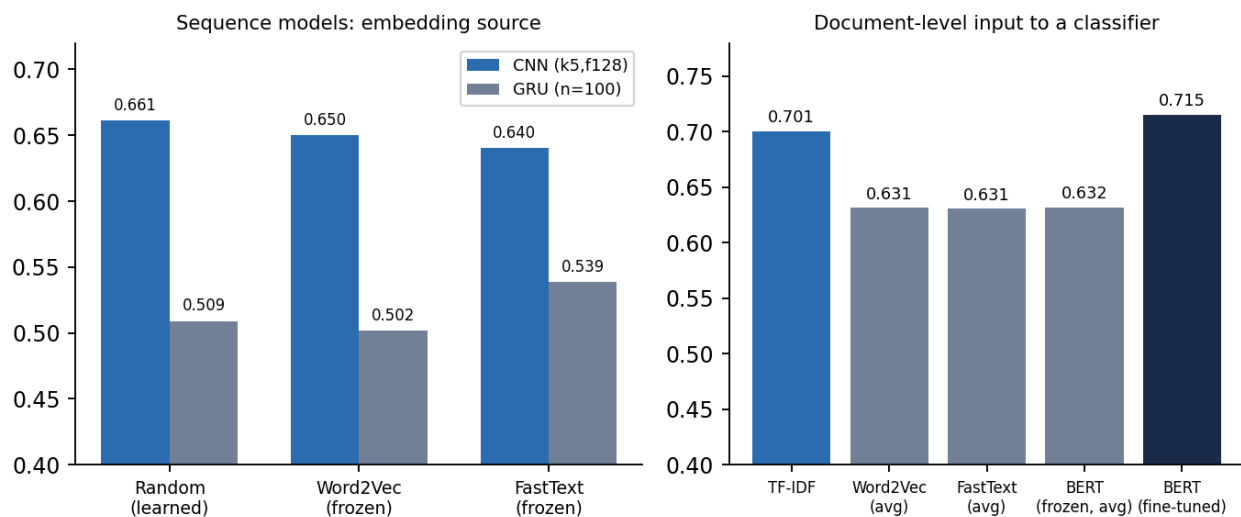


- Reading 100 tokens beat reading 10 for every model except SimpleRNN. The best RNN, BiLSTM at n = 100 (0.5752), is still 12.6 points below the TF-IDF ANN.
- Bidirectional versions beat one-direction versions at n = 100: BiLSTM 0.5752 vs LSTM 0.4538; BiGRU 0.5536 vs GRU 0.5090.
- SimpleRNN at n = 100 (0.0651) is at chance level. This fits the known vanishing-gradient weakness of plain RNNs, but the training curves were not inspected to confirm it.

5. Pretrained embeddings (Task 5)

Word2Vec (Google News, 300 dimensions) and FastText (Wiki News subwords, 300 dimensions) were plugged into the CNN (k5, f128) and the GRU (n = 100) as **frozen** embedding layers, and averaged into one vector per post for the ANN. BERT was used frozen, mean-pooled into one 768-dimension vector per post, for the ANN.

Coverage of the 9,999-word vocabulary: Word2Vec 9,184 words found (91.8%), FastText 9,463 found (94.6%). Missing vectors are therefore not the explanation for the results.



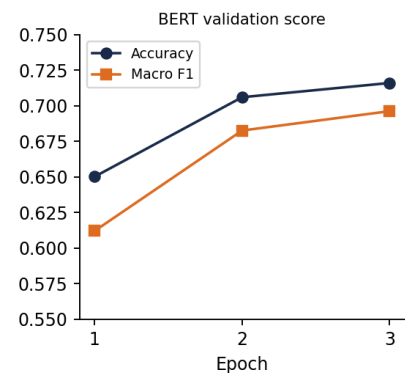
Model	Random (learned)	Word2Vec	FastText	BERT frozen
CNN (k5, f128)	0.6611	0.6498	0.6403	not run
GRU (n = 100)	0.5090	0.5016	0.5387	not run
ANN	0.7007 (TF-IDF)	0.6314	0.6307	0.6321

- Frozen static vectors did not beat learned ones in the CNN (-1.1 and -2.1 points). Only FastText in the GRU gained (+3.0), and Word2Vec there was flat.
- In the ANN all three embeddings (Word2Vec, FastText, BERT) land within 0.14 points of each other, about 7 points below TF-IDF. Averaging a whole post into one vector throws away word identity and order.
- Likely contributors (not isolated by separate experiments): the embedding layers were frozen, and stopwords were already removed from the text.
- **Scope note:** embeddings were run in the ANN, CNN and GRU only. LSTM, BiLSTM, BiGRU and SimpleRNN with embeddings, and BERT features in the CNN and RNNs, were not run.

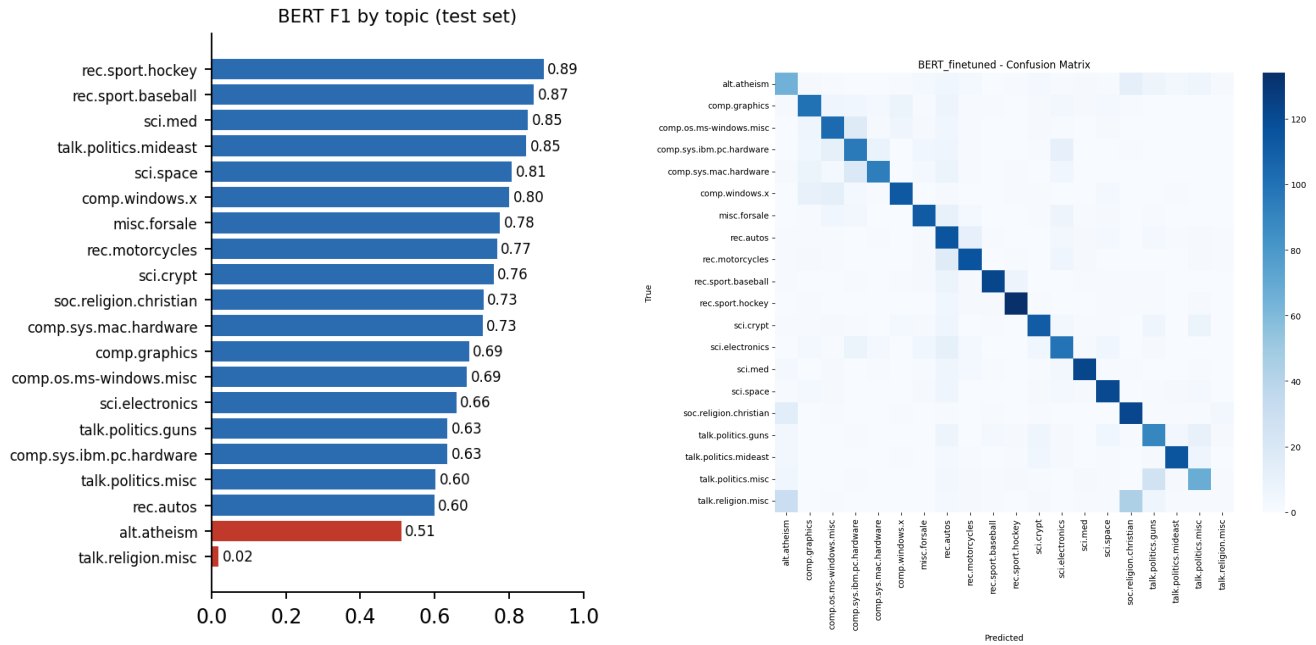
6. Fine-tuned BERT (Task 6)

bert-base-uncased (about 110 million weights) with a new 20-way output layer, all weights updated. Settings: 3 epochs, learning rate 2e-5 with 250 warm-up steps, batch of 16 per GPU on two GPUs (1,239 training steps), maximum 256 tokens, half precision. Training time: 1,045 seconds (about 17 minutes).

Epoch	Validation loss	Validation accuracy	Validation macro F1
1	2.417	0.6502	0.6121
2	1.976	0.7060	0.6826
3 (kept)	1.909	0.7160	0.6962



Test result: accuracy 0.7152, macro precision 0.6956, macro recall 0.6990, macro F1 0.6929 (validation 0.716, so no sign of overfitting to the validation set). The model was still improving and its training loss (about 1.6) was still high, so it is undertrained rather than overfit; a longer schedule was not run.

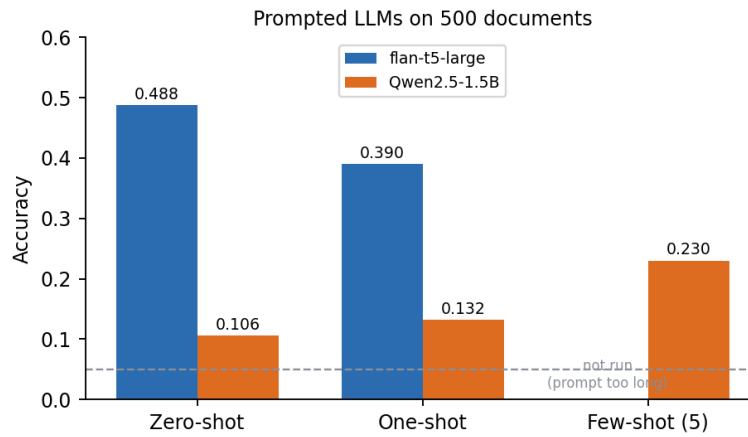


- Best topics: rec.sport.hockey (F1 0.893), rec.sport.baseball (0.866), sci.med (0.851), talk.politics.mideast (0.846): distinctive vocabulary.
- Worst topic: **talk.religion.misc, F1 0.019** (recall 0.011, 1 of 94). It overlaps with alt.atheism (F1 0.510) and soc.religion.christian (0.731).
- rec.autos has the lowest precision among the mid-scoring classes (0.489): other topics are dumped into it (recall 0.772).
- The comp.* groups range from 0.634 to 0.801 and blur into each other.

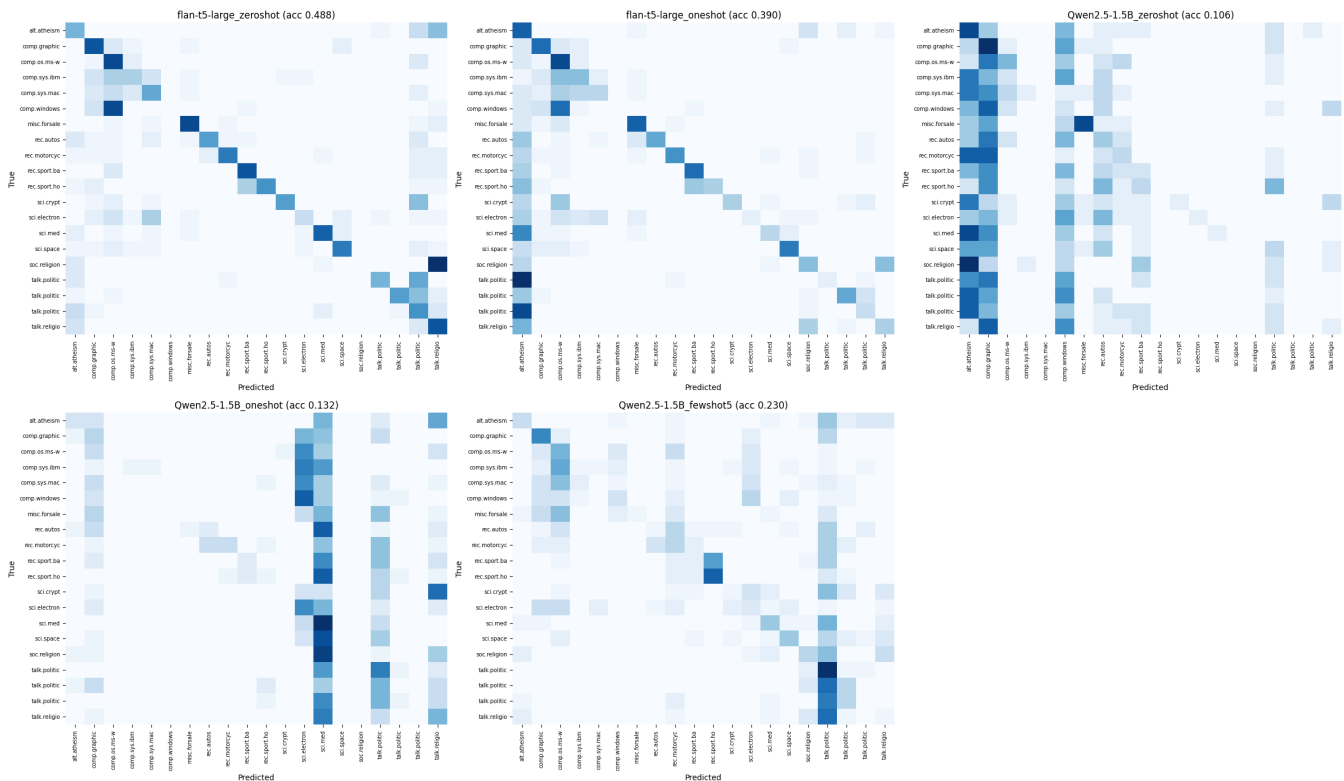
7. Large language models, no training (Task 7)

Each post was shown with a list of the 20 topics labelled A to T, and the answer was the letter the model scored highest (one forward pass per post, no free text). 500 posts, 25 per topic, identical for all runs. One-shot uses 1 example per topic (20 in the prompt, 834 words); few-shot uses 5 per topic (100 examples, 3,211 words, each cut to 40 words).

Run	Accuracy	Macro precision	Macro F1	Topics predicted
flan-t5-large (0.78B) zero-shot	0.4880	0.5658	0.4855	18
flan-t5-large one-shot	0.3900	0.6061	0.4026	19
Qwen2.5-1.5B-Instruct zero-shot	0.1060	0.2341	0.0877	15
Qwen2.5-1.5B one-shot	0.1320	0.1922	0.0889	15
Qwen2.5-1.5B 5-shot	0.2300	0.2810	0.2019	20



- flan-t5 does well with no examples because it was trained on instruction-style tasks. Adding one example per topic **lowered** its accuracy (0.488 to 0.390), most likely because the 834-word prompt is far beyond the roughly 512-token inputs it was trained on. flan-t5 was not run 5-shot for the same reason.
- Qwen improved with more examples (0.106, 0.132, 0.230), but the confusion matrices show most predictions falling on a few topics (for example comp.graphics in zero-shot, sci.med in one-shot). Part of its low score is this letter bias rather than lack of topic knowledge.
- A correction that subtracts each letter's average score (calibration) was attempted but not completed, so it is not reported.
- DistilBERT and RoBERTa were not used in this task because encoder-only models cannot follow a prompt; BERT was covered by fine-tuning in Task 6.



Confusion matrices for the five LLM runs (rows: true topic, columns: predicted).

8. Live test on new sentences

Four sentences written by hand (never seen in training) were cleaned with the same function and classified by the saved fine-tuned BERT checkpoint (checkpoint-1239):

Sentence (short form)	Prediction	Confidence	Other candidates
Yankees traded their closing pitcher	rec.sport.baseball	0.95	hockey 0.01
Good treatment for a persistent migraine	sci.med	0.95	politics.misc 0.01
Graphics card driver crashes on Windows	comp.os.ms-windows.misc	0.71	ibm.pc.hardware 0.10; comp.graphics 0.06
I do not believe in any god...	alt.atheism	0.51	religion.misc 0.26; soc.religion.christian 0.15

- Clear-topic sentences get 95% confidence; overlapping topics split the probability, matching the confusion seen in the test set.
- Cleaning changed the meaning of the last sentence: "I don't believe in any god, and the arguments for religion don't convince me" became "believe god arguments religion convince". The negation was deleted with the stopwords. The model still answered correctly, but this is a real cost of the cleaning step.
- Four hand-written sentences are an illustration, not a measurement.

9. What the numbers mean in practice

If the system sorted 1,000 posts	Correctly filed	Misfiled
Random guessing (20 topics)	about 50	about 950
Fine-tuned BERT	about 715	about 285
TF-IDF ANN	about 701	about 299
Best CNN	about 678	about 322
Best RNN (BiLSTM, n = 100)	about 575	about 425
flan-t5-large, zero-shot (500-post test)	about 488	about 512

- Cost versus gain: BERT needed a GPU run of about 17 minutes and 110 million weights to gain 41 posts out of 2,827 over the TF-IDF network. Whether that is worth it depends on the job; on this dataset it is a small margin.
- Some topics are close to unusable for any model here (talk.religion.misc), because human posters in those groups discuss overlapping things. Accuracy per topic matters more than the overall average when one topic is the one you care about.
- Models that never saw the data (LLMs, prompted) can reach about half the accuracy of a trained model with no training at all, which is useful when no labelled data exists.

10. Limitations and honesty notes

- Stopwords and punctuation were removed for every model. This is a handicap for BERT and the LLMs, which rely on natural text; it likely lowered their scores (untested).
- Single run per model; no repeated seeds, so differences under about 2 points are not conclusive.
- BERT was trained for only 3 epochs; 6 epochs with a higher learning rate was proposed but not run.
- LLM scores use 500 posts (margin about +/-4.4 points); trained-model scores use 2,827.
- Task 5 is a subset of the assignment: Word2Vec and FastText in ANN, CNN and GRU; BERT as frozen features in the ANN only.
- Numbers for Tasks 2 to 4 are taken from the saved results file of the notebook (the written commentary in the notebook for Task 4 is from an earlier run and no longer matches).
- The raw 20 Newsgroups posts contain headers and quoted replies that can leak the topic; the notebook used its own cleaning and this report did not test how much leakage remained.

11. Conclusions

- For topic classification of newsgroup posts, a well-adapted transformer (fine-tuned BERT, 0.715) is best, but a simple TF-IDF network (0.701) is nearly as good at a fraction of the cost.
- Pretrained knowledge helps most when it can adapt to the task (fine-tuning: +8.3 points), and much less when frozen.
- More context helps recurrent models, and plain RNNs can fail completely on long inputs.
- Prompted small LLMs are usable without training data but are sensitive to prompt length and to label-letter bias.

Appendix A. All 32 runs

#	Model	Test set	Accuracy	Macro precision	Macro recall	Macro F1
1	BERT_finetuned	2827	0.7152	0.6956	0.6990	0.6929
2	ANN_TFIDF	2827	0.7007	0.7084	0.6925	0.6961
3	CNN_k7_f128	2827	0.6778	0.6818	0.6688	0.6719
4	CNN_k3_f64	2827	0.6654	0.6695	0.6551	0.6572
5	CNN_k5_f128	2827	0.6611	0.6791	0.6523	0.6567
6	CNN_Word2Vec	2827	0.6498	0.6510	0.6379	0.6390
7	CNN_FastText	2827	0.6403	0.6439	0.6284	0.6256
8	ANN_BERT	2827	0.6321	0.6219	0.6165	0.6111
9	ANN_Word2Vec	2827	0.6314	0.6405	0.6181	0.6183
10	ANN_FastText	2827	0.6307	0.6200	0.6141	0.6094
11	BiLSTM_n100	2827	0.5752	0.5574	0.5601	0.5534
12	BiGRU_n100	2827	0.5536	0.5365	0.5365	0.5255
13	GRU_FastText	2827	0.5387	0.5379	0.5267	0.5230
14	GRU_n100	2827	0.5090	0.4820	0.4958	0.4799
15	GRU_Word2Vec	2827	0.5016	0.4697	0.4873	0.4691
16	flan-t5-large_zeroshot	500	0.4880	0.5658	0.4880	0.4855
17	LSTM_n100	2827	0.4538	0.4248	0.4424	0.4158
18	BiLSTM_n10	2827	0.4362	0.4273	0.4220	0.4120
19	BiGRU_n10	2827	0.4199	0.4197	0.4091	0.4021
20	LSTM_n10	2827	0.4093	0.3852	0.3951	0.3752
21	flan-t5-large_oneshot	500	0.3900	0.6061	0.3900	0.4026
22	SimpleRNN_n10	2827	0.3750	0.3592	0.3629	0.3509
23	GRU_n10	2827	0.3672	0.3439	0.3544	0.3320
24	BiGRU_n5	2827	0.3307	0.3248	0.3218	0.3121
25	BiLSTM_n5	2827	0.3247	0.3236	0.3149	0.3072
26	GRU_n5	2827	0.2989	0.2825	0.2880	0.2755
27	LSTM_n5	2827	0.2947	0.2873	0.2864	0.2753
28	SimpleRNN_n5	2827	0.2547	0.2562	0.2454	0.2288
29	Qwen2.5-1.5B_fewshot5	500	0.2300	0.2810	0.2300	0.2019
30	Qwen2.5-1.5B_oneshot	500	0.1320	0.1922	0.1320	0.0889
31	Qwen2.5-1.5B_zeroshot	500	0.1060	0.2341	0.1060	0.0877
32	SimpleRNN_n100	2827	0.0651	0.0327	0.0642	0.0182

Test set = number of test posts the run was scored on (2827 full test set; 500 LLM subsample).

Appendix B. Accuracy of every run

